

StyleScript: A Structured Data Augmentation Framework for Transformer-Based OCR in Engineering Documents

Farshid Javadnejad
Old Dominion University
fjava001@odu.edu

Jiang Li
Old Dominion University
jli@odu.edu

Samuel Kovacic
Old Dominion University
skovacic@odu.edu

Andres Sousa-Poza
Old Dominion University
asousapo@odu.edu

Wael Khallouli
Old Dominion University
wkhallou@odu.edu

Abstract

Engineering documents often contain a combination of printed and handwritten text, intricate layouts, and visual degradation, posing significant challenges to Optical Character Recognition (OCR) systems. Transformer-based models like TrOCR provide strong baseline performance but require domain-specific data augmentation to generalize effectively. This paper introduces StyleScript, a structured data augmentation framework that generates realistic synthetic word images by extracting stroke-based style features such as slant angle and thickness. We fine-tune both TrOCR (small) and TrOCR (large) models using real and StyleScript-augmented data derived from a Military Sealift Command (MSC) dataset. Additionally, we develop a systematic OCR pipeline combining preprocessing, CRAFT-based text detection, and fine-tuned TrOCR recognition to digitize full-page engineering documents with spatial fidelity. Experimental results show that StyleScript-enhanced training improves OCR performance across diverse document conditions, making it a practical solution for engineering and other technical domains with limited annotated data.

Keywords: Optical Character Recognition, Data Augmentation, Transformer Models, Digital Engineering, Style Transfer.

1. Introduction

Systems engineering is increasingly shifting from traditional document-based methods to more agile, integrated approaches to address the complexity of modern systems. Model-Based Systems Engineering (MBSE) supports this shift by offering a structured approach to capturing, analyzing, and designing system architectures and processes. It provides a unified modeling framework that enhances collaboration and interoperability across stakeholders and lifecycle phases (Saeidi et al., 2024). Digital engineering builds on this by connecting models, data,

and processes throughout the lifecycle. Together, MBSE and digital engineering aim to improve efficiency, adaptability, and decision-making in complex systems

Despite this transformation, many engineering domains—especially in shipbuilding, aerospace, and manufacturing—remain reliant on legacy artifacts like scanned documents and handwritten annotations. These unstructured records contain critical information but resist integration into model-based workflows. Unlocking their value requires automated digitization to convert them into machine-readable, model-compatible formats.

Optical Character Recognition (OCR) is central to digitizing content from technical drawings, maintenance records, and other engineering documents. While OCR works well on clean printed text, engineering documents present challenges due to stylistic variation, handwritten elements, degraded quality, and intricate layouts (Khallouli et al., 2024). Conventional OCR systems struggle with this variability, and scarce annotated datasets further limit deep learning models' generalization. Data augmentation is needed to simulate diverse styles and improve model robustness. Enhancing OCR accuracy is essential for integrating document-derived data into MBSE and digital engineering ecosystems, where traceability and reusability are crucial.

The main challenge stems from mixed text types. Printed fonts are standardized, but handwritten annotations vary in stroke, slant, and ligatures, requiring large datasets to model (Pippi et al., 2023). Many documents also suffer from poor scans and noisy backgrounds (Villena Toro et al., 2023). Public datasets rarely include domain-specific features like technical terms or symbols, limiting pretraining applicability. These issues, combined with model complexity, highlight the need for targeted augmentation.

Transformer architectures offer a solution. TrOCR combines a Vision Transformer (ViT) encoder with a transformer-based decoder, enabling end-to-end recognition without recurrent layers (Li et al., 2023). It excels in engineering contexts but depends

on diverse training data. Traditional augmentations like geometric transformations and noise injection are simple but fail to reflect the nuance of mixed handwritten and printed engineering text (Wigington et al., 2017). GANs improve on this by generating synthetic images. ScrabbleGAN (Fogel et al., 2020) produces variable-length text, improving OCR with TrOCR and KerasOCR but struggles with style (Gan et al., 2022). HiGAN+ addresses style-content disentanglement but is computationally costly (Gan et al., 2022; Jemni et al., 2022).

Inspired by HiGAN+, we propose StyleScript, a deterministic augmentation method for engineering documents. It uses OpenCV-based processing to extract style features (stroke thickness, slant angle) and render synthetic text in mixed styles. Unlike GANs, it avoids adversarial training, ensuring efficiency, and generates up to 10,000 synthetic images. Our experiments use the Military Sealift Command (MSC) dataset with diverse styles and noise. We evaluate small and large TrOCR variants with generated samples and benchmark against ScrabbleGAN and other OCR tools, Tesseract (<https://github.com/tesseract-ocr/tesseract>), EasyOCR (<https://github.com/JaidedAI/EasyOCR>), KerasOCR (<https://keras-ocr.readthedocs.io/en/latest>), using Handwriting Distance (HWD) and Character Error Rate (CER). This research offers a targeted augmentation solution to mitigate OCR limitations in engineering document analysis.

2. Related Work

Transformer architectures have significantly advanced OCR and Handwritten Text Recognition (HTR), particularly for domains like engineering. The Transformer-based Optical Character Recognition (TrOCR) model, developed by Microsoft (Li et al., 2023), integrates a Vision Transformer (ViT) encoder with a RoBERTa-based textual transformer decoder, leveraging self-attention to capture contextual dependencies for text recognition. Following a typical encoder-decoder architecture, TrOCR uses its visual transformer for feature extraction and its textual transformer for text recognition, having been pre-trained on large datasets of both printed and handwritten text. TrOCR excels on datasets like IAM and SROIE, though its performance relies on diverse training data (Li et al., 2023). Recent extensions include Ullah et al. (2025), who combined TrOCR with Roboflow for handwritten prescription recognition, achieving a WER of 12.5% and CER of 8.7% in clinical settings, and Ambata et al. (2025), who employed sequence-to-sequence models with attention for form extraction, enhancing robustness to

unconstrained handwriting. For non-Latin scripts, Al-Maamari et al. (2025) proposed a hybrid transformer encoder with VGG16 and ResNet50 pretraining for Arabic printed and handwritten characters.

Koch et al. (2023) proposed a transformer-based HTR system for medieval Latin manuscripts, combining ViT, BEiT, and Swin Transformer encoders with a GPT decoder, improving performance on stylized scripts but requiring extensive fine-tuning. Hamdan et al. (2024) introduced HTR-JAND, a transformer-based model using joint attention networks and knowledge distillation for historical documents across languages, effectively modeling diverse styles despite computational challenges. Kang et al. (2022) explored attention-based CNN models for HTR, while Easter2.4 combined convolutional and transformer layers, though struggling with irregular structural arrangements (Chaudhary & Bali, 2022). Khallouli et al. (2024) demonstrated improved recognition of engineering documents using transformer-based OCR with generative augmentation. These studies highlight transformers' dominance in OCR and HTR yet underscore the need for augmentation in data-scarce domains.

2.1. Data Augmentation for Printed and Handwritten Text Recognition

Data augmentation addresses the scarcity of annotated datasets in OCR and HTR. Conventional methods like geometric transformations and noise injection enhance robustness but fail to capture stylistic diversity (Wigington et al., 2017). Generative Adversarial Networks (GANs) offer a solution. ScrabbleGAN generates variable-length text images, improving recognition for models like TrOCR and KerasOCR, though lacking style control (Fogel et al., 2020). Eltay et al. (2022) used GAN-based augmentation to synthesize handwritten text, enhancing HTR generalization. Rabi and Amrouche (2024) developed a GAN-augmented CNN-BLSTM model for HTR, boosting performance but facing computational complexity. Koch et al. (2023) applied augmentation for medieval Latin HTR with rotations and noise. Hamdan et al. (2024)'s HTR-JAND used GAN-based data generation with curriculum learning. GANwriting conditions handwritten word generation on style and content, supporting out-of-vocabulary (OOV) text but limited by style diversity (Kang et al., 2020). SLOGAN synthesizes arbitrary-length OOV text using a style bank (Luo et al., 2022). HiGAN+ disentangles content and style, generating high-quality handwritten text despite training complexity (Gan et al., 2022). A text-and-speech-conditioned GAN generates handwriting lines (Davis et al., 2020).

Cascianelli et al. (2022)’s deformable convolution-based HTR reduced augmentation reliance. Diff-Font, a diffusion model, generates Chinese and Korean characters for OCR training (He et al., 2024). TextDiffuser generates text images using a Layout Transformer and diffusion model (Chen et al., 2023). FontRNN focuses on printed text (Luo et al., 2020), while Bhunia et al. (2021)’s Handwriting Transformers mimic scripts at high computational cost.

2.2. Document Enhancement and Binarization

Preprocessing, such as binarization and enhancement, is critical for noisy engineering documents. Jemni et al. (2022) proposed a GAN-based enhancement framework to improve readability on degraded texts. Koch et al. (2023) addressed scanning artifacts in medieval Latin manuscripts. Deformable convolution models required minimal preprocessing by focusing on ink pixels (Cascianelli et al., 2022). A deep learning pipeline for historical documents improved HTR performance (Souibgui et al., 2021). Hamdan et al. (2024)’s HTR-JAND used contrast normalization for faded texts. These works emphasize deep learning’s role in preprocessing, balancing cost and generalizability.

2.3. Styled Text Generation for Printed and Handwritten Text

Styled Text Generation (STG) creates diverse OCR and HTR training data. Pippi et al. (2023) introduced Handwriting Distance (HWD) for style fidelity evaluation on IAM, advancing STG with their Visual Archetypes-based Transformer (VATr). HiGAN+ employs content and style encoders with contextual loss for realistic handwritten text (Gan et al., 2022). Koch et al. (2023) mimicked historical scripts for medieval Latin HTR. GANwriting generates style-conditioned handwritten words (Kang et al., 2020). SLOGAN synthesizes OOV text using a style bank (Luo et al., 2022). Text-and-speech-conditioned GANs generate handwriting lines (Davis et al., 2020). Hamdan et al. (2024)’s HTR-JAND used attention for style-aware generation. Cascianelli et al. (2022)’s deformable convolution models adapted kernels to handwriting. Diff-Font generates high-fidelity Chinese and Korean fonts (He et al., 2024). TextDiffuser renders text images with a Layout Transformer (Chen et al., 2023). Kang et al. (2022) explored CycleGAN-based style transfer.

3. Methodology

3.1. StyleScript

HiGAN, introduced by Gan and Wang (2021), is a GAN for Handwriting Imitation (HI), generating realistic handwriting images conditioned on arbitrary text and styles, including OOV words, using disentangled representations to separate content and style for one-shot style transfer, supporting variable-sized outputs via character-level embeddings, though often producing blurred characters. HiGAN+, proposed by Gan et al. (2022), enhances this with contextual loss for style consistency, Local Patch Loss (LPL) to reduce distortions, and compact style encoding via writer identifier reuse, resulting in a smaller, more realistic model compared to GANs like ScrabbleGAN (Fogel et al., 2020) and GANwriting (Kang et al., 2020).

StyleScript’s framework (Figure 1.) starts with Phase 1: the Style Feature Extraction Module $\Phi(x)$, extracting calligraphic features from grayscale reference images. For stroke thickness analysis, the algorithm first converts the input image to grayscale and applies binary thresholding to isolate the handwritten strokes from the background. Using OpenCV’s contour detection algorithms (OpenCV), all connected components are identified and filtered to retain only meaningful stroke segments with area greater than 10 pixels. The stroke thickness is then computed as the mean area-to-perimeter ratio across all valid contours. The slant angle detection process employs Canny edge detection followed by Hough Line Transform (Al Noman et al., 2023; Bradski, 2000; Hiremath & Shreenidhi; OpenCV) to identify the dominant directional patterns in the handwriting, forming a style vector, though less robust than HiGAN+’s neural encoding.

$$\tau = \left(\frac{1}{N}\right) \sum_{i=1}^N \frac{A_i}{P_i} \quad (1)$$

The stroke thickness measure, denoted as τ , is calculated by evaluating the average area-to-perimeter ratio of valid contours in a handwriting image. Here, N represents the number of valid contours, which are defined as those with an area greater than 10 pixels. For each contour, A_i is the area of the i^{th} contour obtained using `cv2.contourArea()`, and P_i is its corresponding perimeter, computed via `cv2.arcLength()`.

$$\theta = \left(\frac{1}{M}\right) \sum_{j=1}^M \arctan2(y_{2j} - y_{1j}, x_{2j} - x_{1j}) \times \left(\frac{180}{\pi}\right) \quad (2)$$

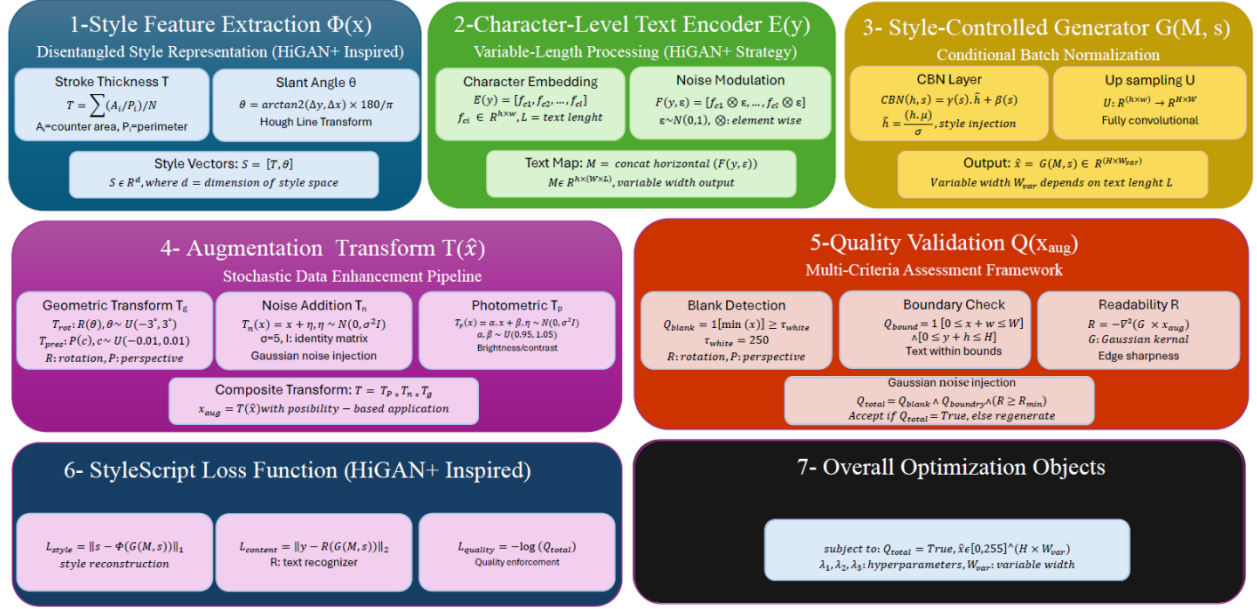


Figure 1. Framework of the proposed StyleScript OCR data augmentation method.

The slant angle, denoted as θ , is measured in degrees and is derived from the orientation of lines detected in the handwriting image. M represents the total number of lines identified using `cv2.HoughLinesP()`. For each detected line j , the endpoints are represented as (x_{1j}, y_{1j}) and (x_{2j}, y_{2j}) , which define the spatial coordinates used to calculate the slant angle.

$$s = \Phi(x) = [T, \theta] \in \mathbb{R}^d \quad (3)$$

The complete style vector is denoted as s and is obtained by applying the style extraction function $\Phi(x)$ to the input handwriting image x . This function captures the relevant stylistic features present in the image. The resulting vector resides in a feature space of dimensionality d , which defines the number of style attributes encoded.

Phase 2: the character-level text encoder $E(y)$ encodes text into embeddings, applies noise modulation, and concatenates them into a variable-width text map, enabling flexible text length handling. $E(y) = [f_{c1}, f_{c2}, \dots, f_{cL}]$ (4)

The text encoding function is denoted as $E(y)$, where $y = [y_1, y_2, \dots, y_L]$ represents the input text sequence of total length L . For each character c_i in the sequence, a corresponding filter map f_{ci} is generated.

$$F(y, \epsilon) = [f_{c1} \otimes \epsilon, f_{c2} \otimes \epsilon, \dots, f_{cL} \otimes \epsilon] \quad (5)$$

$$M = \text{concat horiz}(F(y, \epsilon)) \in \mathbb{R}^{h \times (w \times L)} \quad (6)$$

Where noise-modulated character embeddings are defined as $F(y, \epsilon)$, where $\otimes$ denotes element-wise multiplication and ϵ is a noise vector sampled from a standard normal distribution. These embeddings are then horizontally concatenated to form the final text map M , resulting in a representation of size $\mathbb{R}^{h \times (w \times L)}$,

where h and w are the height and width of the character maps and L is the total text length.

Phase 3: the style-controlled generator $G(M, s)$ renders text into 128×64 grayscale images. In HiGAN+ (Gan et al., 2022), Conditional Batch Normalization (CBN) modulates feature maps with style information, ensuring style consistency across layers, as defined by the equation below. The style vector s (containing τ and θ) directly adjusts rendering parameters: font size is scaled based on stroke thickness τ , and text is sheared using slant angle θ , producing output images deterministically.

$$CBN(h, s) = \gamma(s) \cdot \frac{(h, \mu)}{\sigma} + \beta(s) \quad (7)$$

$$\text{Font Size Adj Factor} = \min(\max(\frac{\tau}{2}, 0.8), 1.2) \quad (8)$$

$$\text{Shear Trans Matrix} = \begin{bmatrix} 1 & \tan(\min(\max(\theta, -30^\circ), 30^\circ)) & 0 \\ 0 & 1 & 0 \end{bmatrix} \quad (9)$$

$$\hat{x} = G(M, s) \in \mathbb{R}^{H \times W_{var}} \quad (10)$$

Where $CBN(h, s)$ is the conditional batch normalization output, h is the input feature map, s is the style vector, $\gamma(s)$ and $\beta(s)$ are style-dependent scale and shift parameters, μ and σ are batch mean and standard deviation; Font Size Adj Factor is the scaling multiplier for base font size with τ as stroke thickness measure, 0.8 and 1.2 as minimum and maximum scaling bounds; Shear Trans Matrix is the 2D transformation matrix with θ as slant angle constrained between -30° and 30° , and $\tan()$ computing horizontal shearing; $\hat{x}$ is the final generated handwriting image from generator function G , M is the variable-width text map, H is fixed image height (64 pixels), and W_{var} is variable width proportional to text length.

Phase 4: augmentation transform $T_{rot}(\hat{x})$ applies transformations to enhance realism, ensuring robust

outputs in linear time. Here, $T_{rot}(\hat{x})$ rotates the image by a random angle to simulate natural hand variations, $T_{pers}(\hat{x})$ applies perspective warp to mimic viewing angle distortions, T_{noise} adds Gaussian noise for texture variations, and $T_{photo}(\hat{x})$ adjusts brightness and contrast to simulate lighting changes.

$$T_{rot}(\hat{x}) = R(\theta) \cdot \hat{x} \quad (11)$$

$$R(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}, \quad \theta \sim U(-3^\circ, 3^\circ) \quad (12)$$

$$T_{pers}(\hat{x}) = P(c) \cdot \hat{x}, \quad (13)$$

$$P(c) = \begin{bmatrix} 1 & c_1 & 0 \\ c_2 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad c_1, c_2 \sim U(-0.01, 0.01) \quad (12)$$

$$T_{noise}(\hat{x}) = \hat{x} + \eta, \quad \eta \sim N(0, \sigma^2) \quad (13)$$

$$T_{photo}(\hat{x}) = \alpha \cdot \hat{x} + \beta, \quad \alpha, \beta \sim U(0.95, 1.05) = \text{contrast and brightness factors} \quad (14)$$

$$x_{aug} = T(\hat{x}) = T_{rot}(\hat{x}) \circ T_{pers}(\hat{x}) \circ T_{noise}(\hat{x}) \circ T_{photo}(\hat{x}) \quad (15)$$

Where $\hat{x}$ is the input text image, θ is the rotation angle sampled from a uniform distribution $U(-3^\circ, 3^\circ)$, and $R(\theta)$ is the corresponding 2D rotation matrix. c_1 and c_2 are perspective distortion coefficients sampled from $U(-0.01, 0.01)$, forming the transformation matrix $P(c)$. η is Gaussian noise sampled from $N(0, \sigma^2)$, and α and β are brightness and contrast factors sampled from $U(0.95, 1.05)$.

Phase 5: quality validation ($Q(x_{aug})$) ensures image quality by evaluating multiple criteria, rejecting invalid samples to maintain output reliability:

$$Q_{blank} = 1[\min(x_{aug}) \geq \tau_{white}(250)] \quad (14)$$

$$Q_{bound} = 1[0 \leq x + w \leq W] \wedge 1[0 \leq y + h \leq H] \quad (15)$$

$$R = -\nabla^2(G * x_{aug}), \quad G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\pi\sigma^2}} \quad (16)$$

$$Q_{total} = Q_{blank} \wedge Q_{bound} \wedge (R \geq R_{min}) \quad (17)$$

Where x_{aug} is the augmented image, τ_{white} is a threshold (e.g., 250) for blank detection, and Q_{blank} checks if the image is blank. Q_{bound} ensures the bounding box stays within the image dimensions W and H . Edge sharpness is computed as R , where $G(x, y)$ is a Gaussian kernel. The final quality score is Q_{total} , verifying the image is non-blank, properly bounded, and sufficiently sharp.

Phase 6: the StyleScript loss function combines style consistency, content preservation, and quality penalty.

$$\mathcal{L}_{style} = \|s - \Phi(G(M, s))\|_1 \quad (18)$$

$$\mathcal{L}_{content} = \|y - R(G(M, s))\|_2 \quad (19)$$

$$\mathcal{L}_{quality} = -\log(Q_{total}) \quad (20)$$

Where s is the target style vector, y is the target text, M is the encoded text map, $G(M, s)$ is the generated image, $\Phi(G(M, s))$ is the style extraction function, and $R(G(M, s))$ is the recognition model. The style loss is defined as $\mathcal{L}_{style}$, which measures the L1 norm (i.e., the sum of absolute differences) between the target

and generated style features. The content loss, $\mathcal{L}_{content}$, uses the L2 norm (i.e., the square root of the sum of squared differences) to evaluate how closely the generated image matches the target text. The quality loss is $\mathcal{L}_{quality}$, which penalizes images that fail blankness, bounding, or sharpness checks.

Phase 7: the optimization objective balances these losses under constraints

$$\min \mathcal{L}_{total} = \lambda_1 \cdot \mathcal{L}_{style} + \lambda_2 \cdot \mathcal{L}_{content} + \lambda_3 \cdot \mathcal{L}_{quality} \quad (21)$$

3.2. OCR Pipeline

As a practical solution, we developed an OCR pipeline to process full-page engineering documents and output recognized text in digital formats like plain text or JSON, with spatial positioning preserved. The pipeline consists of two interconnected streams: a systematic implementation for document processing and a model development process for training and evaluation. The implementation begins with preprocessing, enhancing document quality through cleaning, binarization, and contrast adjustments to mitigate noise and degradation in engineering documents. This ensures optimal input for downstream steps. Next, the CRAFT (Baek et al., 2019) text detection module identifies text regions and generates bounding boxes, facilitating precise localization even in complex layouts with tables, drawings, or annotations.

The text recognition module uses Microsoft TrOCR (Li et al., 2023), a transformer-based architecture that converts cropped text regions into digital output. It integrates a BEiT-based Vision Encoder (Bao et al., 2021) and a RoBERTa-based Text Decoder (Liu et al., 2019), using attention mechanisms to focus on relevant image regions for improved recognition. We trained this module using manually annotated Ground Truth Data from MSC maintenance documents, evaluating two TrOCR variants. The small version (<https://huggingface.co/microsoft/trocr-small-printed>) includes 62M parameters and uses a DeiT-small visual encoder (12 layers, 384 hidden sizes, 6 heads) with a MiniLM decoder (6 layers, 256 hidden sizes, 8 heads), optimized for efficiency. The large variant <https://huggingface.co/microsoft/trocr-large-printed> contains 558M parameters, featuring a BEiT-large encoder (24 layers, 1024 hidden sizes, 16 heads) and a RoBERTa-large decoder (12 layers, 1024 hidden sizes, 16 heads), yielding superior performance.

To improve robustness and evaluate data augmentation, we added synthetic images generated by StyleScript and ScrabbleGAN (Fogel et al., 2020),

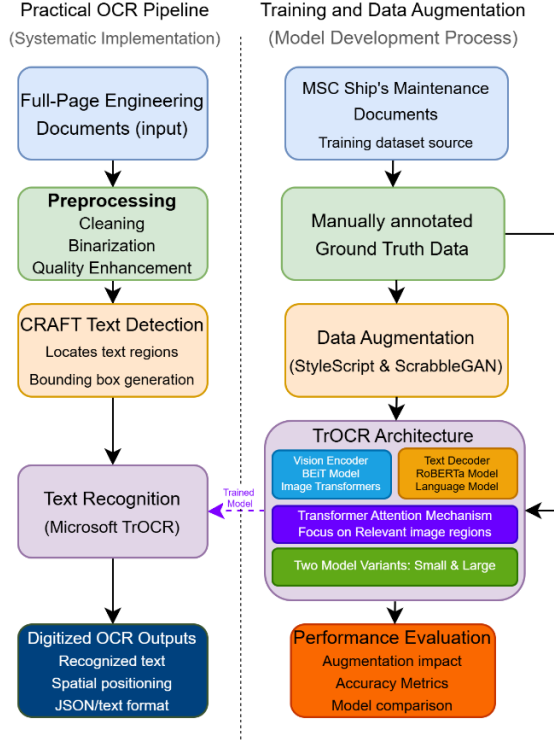


Figure 2. Systematic implementation and model development process for OCR Pipeline.

producing 10,000 samples mimicking font styles and noise in the MSC dataset. This enriched the limited annotated data and supported better generalization. Both synthetic and real data were used to train the TrOCR models, producing digitized outputs with spatial accuracy in JSON or plain text.

Pipeline performance was evaluated based on augmentation impact, accuracy metrics (e.g., CER, WER), and comparisons with baseline models. Figure 2 illustrates this enhanced workflow, showing the interaction between implementation (preprocessing, detection, recognition) and development (annotation, augmentation, evaluation). The left-side vertical pipeline outlines our scalable system, trained on MSC data but adaptable to other domains like aerospace or manufacturing. Its modular structure supports future integration of advanced preprocessing techniques tailored to diverse document types.

3.2. Evaluation Metrics

To evaluate the performance of each component, including StyleScript data augmentation, TrOCR training, and the practical OCR pipeline, a set of quantitative and qualitative metrics are employed to assess accuracy, style fidelity, and applicability.

3.3.1. For StyleScript Data Augmentation. Handwriting Distance (HWD) (Pippi et al., 2023)

measures style fidelity between synthetic and MSC document features. Lower HWD indicates better style preservation.

$$HWD = \sqrt{\sum_{i=1}^N (f_{i,synth} - f_{i,ref})^2} \quad (22)$$

where f_i are style features, N is feature count. Fréchet Inception Distance (FID) (Yu et al., 2021) Assesses synthetic image quality vs. reference MSC images. Lower FID indicates higher realism.

$$FID = \|\mu_{synth} - \mu_{real}\|_2^2 + Tr(\Sigma_{synth} + \Sigma_{real} - 2\sqrt{\Sigma_{synth}\Sigma_{real}}) \quad (23)$$

where μ_{synth} and μ_{real} are the mean feature vectors of the synthetic and real image distributions, respectively, and Σ_{synth} and Σ_{real} are their corresponding covariance matrices. $\|\cdot\|_2^2$ is the squared L2 norm (Euclidean distance squared between means μ_{synth} and μ_{real}).

3.3.2. For TrOCR Training and Testing. Character Error Rate (CER) measures character errors vs. ground truth. Lower CER shows better accuracy.

$$CER = \frac{n_{errors}}{n_{characters}} \times 100 \quad (24)$$

Where n_{errors} is the number of incorrect characters in the recognized text, and $n_{characters}$ is the total number of characters in the reference text.

Word Error Rate (WER) measures word errors vs. ground truth. Lower WER indicates better performance.

$$WER = \frac{n_{errors}}{n_{words}} \times 100 \quad (25)$$

Levenshtein distance measures the minimum edits needed to transform one word into another. We averaged this across all predicted–actual word pairs in the test set. Lower value shows closer match.

$$AVR. Lev = \frac{1}{N_{test}} \sum_{i=1}^{N_{test}} Lev(w_i^{pred}, w_i^{actual}) \quad (26)$$

Where N_{test} is the total number of test samples, w_i^{pred} is the predicted word for the i^{th} sample, and w_i^{actual} is the corresponding ground truth word.

4. Dataset

The dataset, manually curated from ships' maintenance documents supplied by the Military Sealift Command (MSC), comprises 2D ship drawings and textual content detailing critical design and maintenance information. Each document features hundreds of lines showcasing a range of textual styles and fonts typical of ships' maintenance records, including both low-quality and high-quality samples. These documents present diverse textual styles and occasionally contain fonts that OCR systems struggle to recognize, posing significant challenges to achieving high recognition accuracy.

		REVISION
ZONE	REV	DESCRIPTION
SH 8	A	DETAIL OF HAND GRAB ON BILGE WELL STRAIGHT
60-C	A	PLATE MODIFIED
	A-9	BILLS OF MATERIAL MODIFIED TO SUIT
SH 24	B1	DET 46-B, FLOOR CROPPED BACK TO
55-C	B1	CLEAR B.M. LONG'S #2 FACE IS ADDED
SH 6	B2	INSERT R ADDED FR 76 (E.D. #019)
ALL	B3	RE ZONE DRG
SH 2	B4	MODIFIED BLEEDER PLUG WELD
ALL	B5	NAVSEA DWG N# CHANGED
SH 3	B6	DETAIL CODE ADDED FR 79 BETWEEN 11P & 11S
SH 5	B7	CABLE SLOT REMOVED FR 76, 77 & OIL
58-C	B7	STOPS ADDED FR 77 (E.D. #018)
SH 5	B8	CABLE SLOT ADDED FR 76, 77 & OIL
38-C	B8	STOPS ADDED IN LONG'S AT FR 77
SH 5	B9	REFERENCE TO LADDER CLIPS REMOVED
SH 1	B10	ITEMS 1 & 2 : TITLES CHANGED
38-A	B10	ITEM 2 - NAVSEA N# CHANGED
SH 1	B11	ITEMS 4, 5, 6 & 7 ADDED
22-C	B11	REF. N#S ADDED TO MANHOLES NOTE
SH 2	B12	DELETED
16-A	B12	LIFTING EYE DESIGNATED TEMPORARY
	B13	B.M. NAVSEA N# MODIFIED

Figure 3. An example of an MSC document.

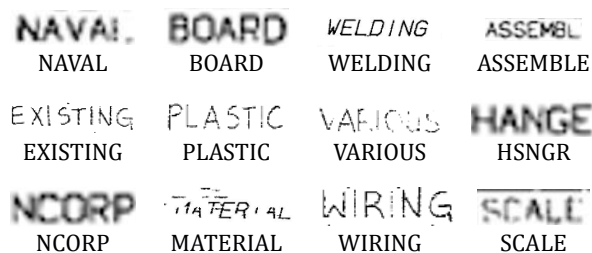


Figure 4. Examples of annotated word images from MSC documents.

We annotated word-level images cropped from the MSC engineering documents, with examples illustrated in Figure 4. The resulting dataset includes 3138-word images, capturing various fonts and styles prevalent in the documentation. This dataset was utilized to assess the performance of our proposed OCR models on text with diverse fonts. It was randomly split into a training set (80%, 2510 images) and a testing set (20%, 628 images).

5. RESULTS

First, we evaluate the potential of pre-trained models on our MSC dataset. Table 1 highlights the text recognition competence of these models when applied to the MSC data, measured using WER, CER, and Levenshtein distance metrics. The pre-trained models include Tesseract, a widely adopted OCR tool enhanced with LSTM neural networks to refine text recognition via preprocessing, segmentation, and classification steps; EasyOCR, a Python-based platform utilizing CRAFT for detection and CRNN for

recognition, pre-trained on a blend of natural and synthetic scene images; and TrOCR (small) and TrOCR (large), transformer-based models designed for efficient and robust text processing, respectively. Several pre-trained OCR systems faced difficulties with the dataset, with Tesseract exhibiting the highest error rates (CER of 13.14% and WER of 32.34%), while TrOCR (large) outperformed others with the lowest errors (CER of 3.52% and WER of 12.47%).

Next, we optimized TrOCR models using the MSC dataset alone and enriched them with synthetic data from StyleScript and ScrabbleGAN. ScrabbleGAN proved to enhance performance over its pre-trained state, demonstrating improved recognition accuracy when integrated into the training process (Khallouli et al., 2024). We engineered StyleScript to craft synthetic word images for training, serving as the cornerstone of our innovative approach to emulate the rich textual diversity within the engineering documents. The full annotated dataset trained StyleScript, with images resized to 128×64 pixels (height fixed at 64 pixels, width adjusted to text length), optimized with a batch size of 10 and a learning rate of 0.0002, enabling it to capture a wide array of font styles and annotations. During inference, StyleScript produced 10,000 synthetic word images, with samples featured in Figure 5.

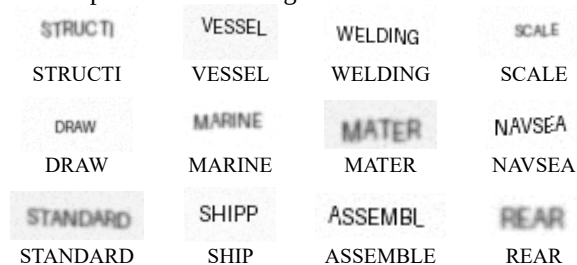


Figure 5. Synthetic samples generated by StyleScript.



Figure 6. Synthetic samples generated by ScrabbleGAN.

Similarly, we trained ScrabbleGAN to create synthetic word images, designed to mimic the textual styles in the documents, using the same dataset with images scaled to $32 \times 16n$ (height set to 32 pixels, width varying with character count n). ScrabbleGAN was tuned with a batch size of 10 and a learning rate of

Table 1. Performance of various models on the MSC testing dataset. ‘MSC’ indicates dataset derived from Military Sealift Command documents, while ‘Aug’ represents the augmented fine-tuning dataset.

OCR Model	Strategy	CER (%)	WER (%)	AVR.Lev
Tesseract	Pre-trained	13.14	32.34	1.2166
EasyOCR	Pre-trained	10.43	30.09	0.6125
KerasOCR	Pre-trained	6.65	21.35	0.3724
TrOCR (small)	Pre-trained	5.54	20.89	0.3041
TrOCR (large)	Pre-trained	3.52	12.47	0.1931
TrOCR (small) w MSC	Fine-tuned	1.81	6.74	0.0972
TrOCR (small) w MSC+ 3000 ScrabbleGAN Aug	Fine-tuned	2.07	6.87	0.1115
TrOCR (small) w MSC+ 10000 ScrabbleGAN Aug	Fine-tuned	2.80	11.02	0.1504
TrOCR (small) w MSC+ 3000 StyleScript Aug	Fine-tuned	1.54	6.48	0.0830
TrOCR (small) w MSC+ 10000 StyleScript Aug	Fine-tuned	2.00	7.78	0.1076
TrOCR (large) w MSC	Fine-tuned	1.40	5.31	0.0752
TrOCR (large) w MSC+ 3000 ScrabbleGAN Aug	Fine-tuned	1.81	6.61	0.0972
TrOCR (large) w MSC+ 10000 ScrabbleGAN Aug	Fine-tuned	2.07	8.43	0.1115
TrOCR (large) w MSC+ 3000 StyleScript Aug	Fine-tuned	1.35	4.79	0.0726
TrOCR (large) w MSC+ 10000 StyleScript Aug	Fine-tuned	1.69	5.83	0.0907

0.0002 across its networks (R, G, and D) and all synthetic word images during inference, with samples shown in Figure 6. The fine-tuning process for TrOCR models used the AdamW optimizer (<https://docs.pytorch.org/docs/stable/generated/torch.optim.AdamW.html>) with a learning rate of 0.00005; the small variant employed a batch size of 32 over 20 epochs, and the large variant used a batch size of 16 over 10 epochs, all executed on an Nvidia A100 high-memory GPU from Old Dominion University (<https://wiki.hpc.odu.edu/en/open-on-demand>).

For TrOCR (small), the baseline model fine-tuned on the MSC dataset alone showed strong performance, with CER reduced to 1.81% and WER to 6.74%. Augmenting with StyleScript improved results further (CER: 1.54%, WER: 6.48%). However, adding 10,000 synthetic images led to performance declines: StyleScript reached a CER of 2.00% and WER of 7.78%, while ScrabbleGAN showed CER of 2.80% and WER of 11.02%, indicating that excessive synthetic data may degrade model effectiveness. For TrOCR (large), fine-tuning on the MSC dataset achieved excellent results (CER: 1.40%, WER: 5.31%). Adding StyleScript images slightly improved performance (CER: 1.35%, WER: 4.79%), while ScrabbleGAN data maintained solid accuracy (CER: 1.81%, WER: 6.61%). Yet, using 10,000 synthetic images again reduced effectiveness, with StyleScript yielding CER of 1.69% and WER of 5.83%, and ScrabbleGAN showing CER of 2.07% and WER of 8.43%. These trends reinforce that over-augmentation remains a challenge and merits further investigation.

We have proposed a systematic and practical OCR pipeline that digitizes and recognizes full-page engineering documents. This pipeline integrates preprocessing, CRAFT-based text detection, and recognition using the fine-tuned TrOCR (small) and

TrOCR (large) models, supporting precise spatial alignment of recognized text. Figure 7 demonstrates the recognition results for the TrOCR (small) and TrOCR (large) models implemented in the proposed OCR pipeline, showcasing their ability to accurately capture annotations, multi-line text, and technical terms while preserving layout fidelity. The performance is attributed to the enriched training set, enhanced with StyleScript-augmented data, which effectively reflects the stylistic and structural diversity of engineering documents.

6. CONCLUSION

This paper introduced StyleScript, a structured data augmentation framework designed to improve OCR accuracy on engineering documents that contain a combination of printed and handwritten text. StyleScript extracts style features such as stroke thickness and slant angle from reference images and applies them through a controlled, modular rendering process to generate realistic, domain-relevant synthetic word images. To evaluate its effectiveness, we fine-tuned two variants of TrOCR model, TrOCR (small) and TrOCR (large) using both the original MSC dataset and synthetic data generated by StyleScript and ScrabbleGAN. Results showed that adding 3,000 StyleScript-generated images led to a consistent performance boost in both models, particularly improving CER and WER compared to the baseline and GAN-based augmentation. However, excessive augmentation diminished performance, highlighting the importance of balanced data diversity. We also developed a practical OCR pipeline tailored for engineering documents, integrating preprocessing, CRAFT-based text detection, and fine-tuned TrOCR recognition to produce spatially aligned outputs in

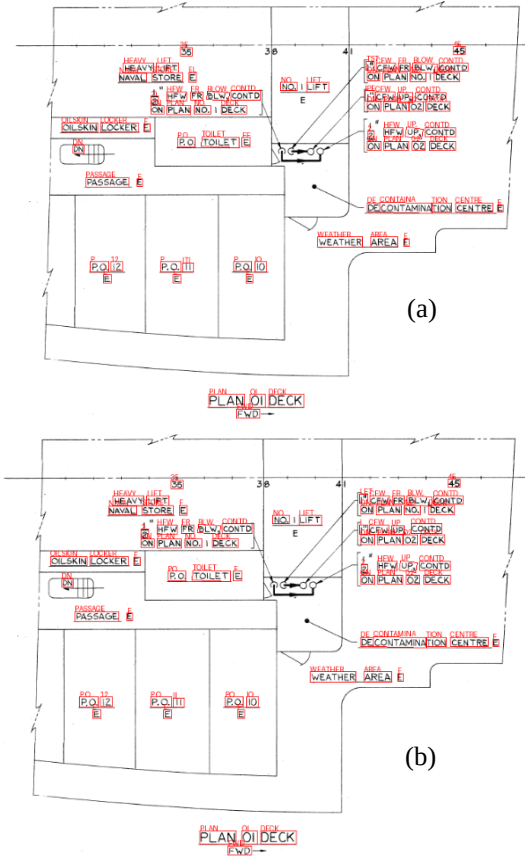


Figure 7. (a) TrOCR (small) with MSC data; (b) TrOCR (large) with 3,000 StyleScript samples.

structured formats such as JSON. This pipeline showed high fidelity in text localization and recognition, especially when using the TrOCR (large) model fine-tuned with StyleScript data, which outperformed other configurations in recognizing technical annotations and multi-line labels. While StyleScript offers an efficient alternative to neural data generation, it simplifies certain style variations and does not adapt to handwriting variability or dataset-specific traits. The observed performance plateau at high augmentation levels suggests limitations in diversity and generalization. We hypothesize that augmentation improves generalization by enriching style diversity, but excessive synthetic data may introduce distributional drift, oversample redundant styles, or dilute real-sample relevance.

Future studies could test this hypothesis to refine strategies and improve robustness, including extending StyleScript to capture richer handwriting characteristics beyond thickness and slant—such as ligature patterns, curvature, and character spacing—to better represent complex writing variations. They could also integrate post-OCR natural language

processing (NLP) techniques tailored to the engineering domain to correct recognition errors using domain-specific syntax, terminology, and context, thereby improving the semantic fidelity of digitized outputs. Moreover, evaluating the approach on larger datasets and diverse types of engineering documents beyond ship maintenance records could help assess its generalizability. Finally, incorporating a lightweight visual analytics interface that overlays detected text regions and recognized outputs on documents could enable users to validate and correct OCR results, while deploying the pipeline in other technical domains such as aerospace, manufacturing, and civil engineering could support domain-specific optimization.

7. References

- Al-Maamari, M. R., Ramteke, R., Al-Hejri, A. M., & Alshamrani, S. S. (2025). Integrating CNN and transformer architectures for superior Arabic printed and handwriting characters classification. *Scientific reports*, 15(1), 29936.
- Al Noman, M. A., Rahaman, M. F., Li, Z., Ray, S., & Wang, C. (2023). A computer vision-based lane detection approach for an autonomous vehicle using the image hough transformation and the edge features. In *Advances in Information Communication Technology and Computing: Proceedings of AICTC 2022* (pp. 53-66). Springer.
- Ambata, L. U., Go, J. D. O., MacaranasIII, P. Q., Ng, S. C., Abayan, J. J. A., Bandala, A. A., Agulto, R. P., Vicerra, R. R. P., Pao, J. C., & Valentos, P. J. R. V. (2025). Automated Extraction of Handwritten Text from Forms Using Advanced Handwritten Text Recognition (HTR). 2025 IEEE International Conference on Automatic Control and Intelligent Systems (I2CACIS),
- Baek, Y., Lee, B., Han, D., Yun, S., & Lee, H. (2019). Character region awareness for text detection. *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*,
- Bao, H., Dong, L., Piao, S., & Wei, F. (2021). Beit: Bert pre-training of image transformers. *arXiv preprint arXiv:2106.08254*.
- Bhunja, A. K., Khan, S., Cholakkal, H., Anwer, R. M., Khan, F. S., & Shah, M. (2021). Handwriting transformers. *Proceedings of the IEEE/CVF international conference on computer vision*,
- Bradski, G. (2000). The OpenCV Library. *Dr. Dobb's Journal of Software Tools*.
- Cascianelli, S., Cornia, M., Baraldi, L., & Cucchiara, R. (2022). Boosting modern and historical handwritten text recognition with deformable convolutions. *International Journal on Document Analysis and Recognition (IJDAR)*, 25(3), 207-217.
- Chaudhary, K., & Bali, R. (2022). Easter2. 0: Improving convolutional models for handwritten text recognition. *arXiv preprint arXiv:2205.14879*.

- Chen, J., Huang, Y., Lv, T., Cui, L., Chen, Q., & Wei, F. (2023). Textdiffuser: Diffusion models as text painters. *Advances in neural information processing systems*, 36, 9353-9387.
- Davis, B., Tensmeyer, C., Price, B., Wigington, C., Morse, B., & Jain, R. (2020). Text and style conditioned GAN for generation of offline handwriting lines. *arXiv preprint arXiv:2009.00678*.
- Eltay, M., Zidouri, A., Ahmad, I., & Elarian, Y. (2022). Generative adversarial network based adaptive data augmentation for handwritten Arabic text recognition. *PeerJ Computer Science*, 8, e861.
- Fogel, S., Averbuch-Elor, H., Cohen, S., Mazor, S., & Litman, R. (2020). Scrabblegan: Semi-supervised varying length handwritten text generation. Proceedings of the IEEE/CVF conference on computer vision and pattern recognition,
- Gan, J., & Wang, W. (2021). HiGAN: handwriting imitation conditioned on arbitrary-length texts and disentangled styles. Proceedings of the AAAI Conference on Artificial Intelligence,
- Gan, J., Wang, W., Leng, J., & Gao, X. (2022). HiGAN+: handwriting imitation GAN with disentangled representations. *ACM Transactions on Graphics (TOG)*, 42(1), 1-17.
- Hamdan, M., Rahiche, A., & Cheriet, M. (2024). HTR-JAND: Handwritten Text Recognition with Joint Attention Network and Knowledge Distillation. *arXiv preprint arXiv:2412.18524*.
- He, H., Chen, X., Wang, C., Liu, J., Du, B., Tao, D., & Yu, Q. (2024). Diff-font: Diffusion model for robust one-shot font generation. *International Journal of Computer Vision*, 132(11), 5372-5386.
- Hiremath, S., & Shreenidhi, B. Dynamic Hough transform for robust lane detection and navigation in real time.
- Jemni, S. K., Souibgui, M. A., Kessentini, Y., & Fornés, A. (2022). Enhance to read better: a multi-task adversarial network for handwritten document image enhancement. *Pattern recognition*, 123, 108370.
- Kang, L., Riba, P., Rusiñol, M., Fornés, A., & Villegas, M. (2022). Pay attention to what you read: non-recurrent handwritten text-line recognition. *Pattern recognition*, 129, 108766.
- Kang, L., Riba, P., Wang, Y., Rusinol, M., Fornés, A., & Villegas, M. (2020). GANwriting: content-conditioned generation of styled handwritten word images. Computer Vision–ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part XXIII 16,
- Khallouli, W., Uddin, M. S., Sousa-Poza, A., Li, J., & Kovacic, S. (2024). Leveraging Transformer-Based OCR Model with Generative Data Augmentation for Engineering Document Recognition. *Electronics*, 14(1), 5.
- Koch, P., Nuñez, G. V., Arias, E. G., Heumann, C., Schöffel, M., Häberlin, A., & Aßenmacher, M. (2023). A tailored Handwritten-Text-Recognition System for Medieval Latin. *arXiv preprint arXiv:2308.09368*.
- Li, M., Lv, T., Chen, J., Cui, L., Lu, Y., Florencio, D., Zhang, C., Li, Z., & Wei, F. (2023). Trocr: Transformer-based optical character recognition with pre-trained models. Proceedings of the AAAI conference on artificial intelligence,
- Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., & Stoyanov, V. (2019). Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*.
- Luo, C., Zhu, Y., Jin, L., Li, Z., & Peng, D. (2022). SLOGAN: handwriting style synthesis for arbitrary-length and out-of-vocabulary text. *IEEE Transactions on Neural Networks and Learning Systems*, 34(11), 8503-8515.
- Luo, C., Zhu, Y., Jin, L., & Wang, Y. (2020). Learn to augment: Joint data augmentation and network optimization for text recognition. Proceedings of the IEEE/CVF conference on computer vision and pattern recognition,
- OpenCV. Contour Features. https://docs.opencv.org/4.x/dd/d49/tutorial_py_contour_features.html
- OpenCV. Hough Line Transform. https://docs.opencv.org/3.4/d9/db0/tutorial_hough_lines.html
- Pippi, V., Quattrini, F., Cascianelli, S., & Cucchiara, R. (2023). HWD: A novel evaluation score for styled handwritten text generation. *arXiv preprint arXiv:2310.20316*.
- Rabi, M., & Amrouche, M. (2024). Enhancing Arabic Handwritten Recognition System Based CNN-BLSTM Using Generative Adversarial Networks. International Conference on Arabic Language Processing,
- Saeidi, A., Tippannavar, S., & Kovacic, S. (2024). Transformation through Technology: Assessing the Feasibility of Digital Ecosystems Using MBSE. 2024 19th Annual System of Systems Engineering Conference (SoSE),
- Souibgui, M. A., Fornés, A., Kessentini, Y., & Tudor, C. (2021). A few-shot learning approach for historical ciphered manuscript recognition. 2020 25th International Conference on Pattern Recognition (ICPR),
- Ullah, H., Tanveer, M., & Jan, A. (2025). Enhancing Handwritten Prescription Recognition with AI-Driven OCR. *Journal of Computing & Biomedical Informatics*.
- Wigington, C., Stewart, S., Davis, B., Barrett, B., Price, B., & Cohen, S. (2017). Data augmentation for recognition of handwritten words and lines using a CNN-LSTM network. 2017 14th IAPR international conference on document analysis and recognition (ICDAR),
- Yu, Y., Zhang, W., & Deng, Y. (2021). Frechet inception distance (fid) for evaluating gans. *China University of Mining Technology Beijing Graduate School*, 3(11).